

代码 <https://github.com/johnma2006/mamba-minimal>

<https://www.cnblogs.com/smartlooli/p/18639878>

<https://www.ibm.com/cn-zh/think/topics/mamba-model>

分词器 tokenizer
'EleutherAI/gpt-neox-20b'

模型 model
'state-spaces/mamba-370m'

'Mamba is the'



tokenizer

tensor([[46, 31834, 310, 253]])

model

probs.shape torch.Size([1, 50280])

取最大概率40归一化

tokenizer

随机采样/贪心采样

tokenizer

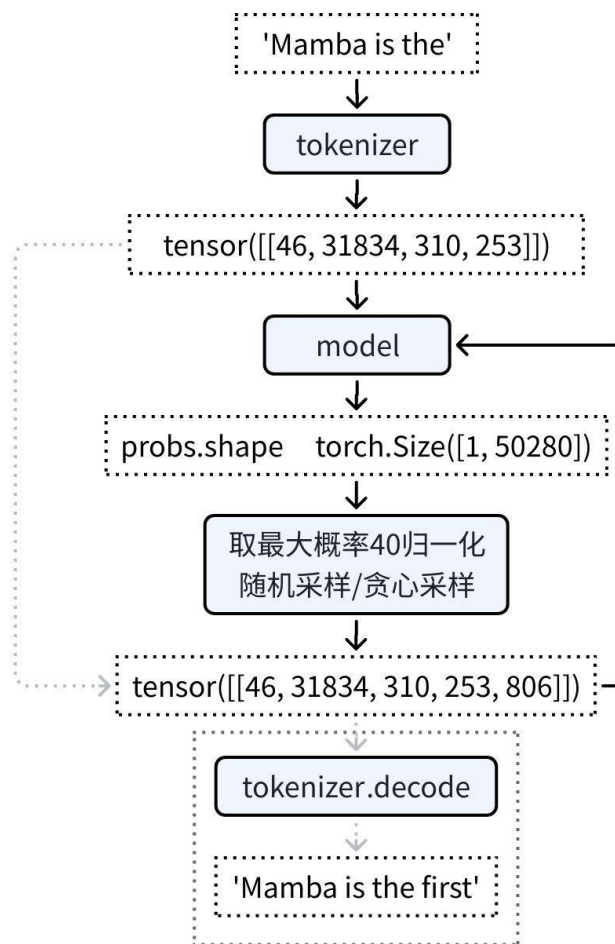
'Mamba is the'

取最大概率40归一化
随机采样/贪心采样

tensor([[46, 31834, 310, 253, 806]])

分词器 tokenizer
'EleutherAI/gpt-neox-20b'

模型 model
'state-spaces/mamba-370m'

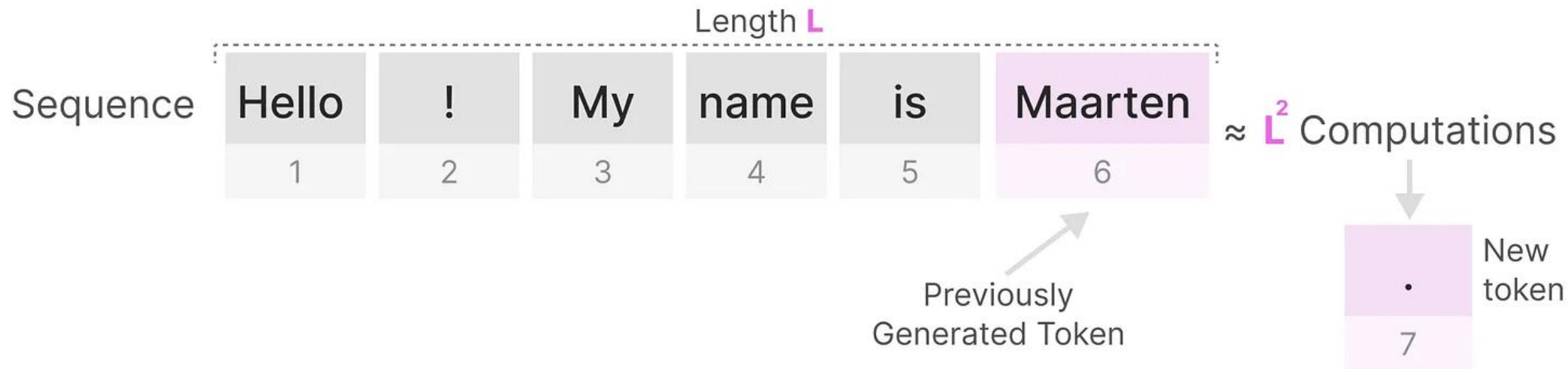


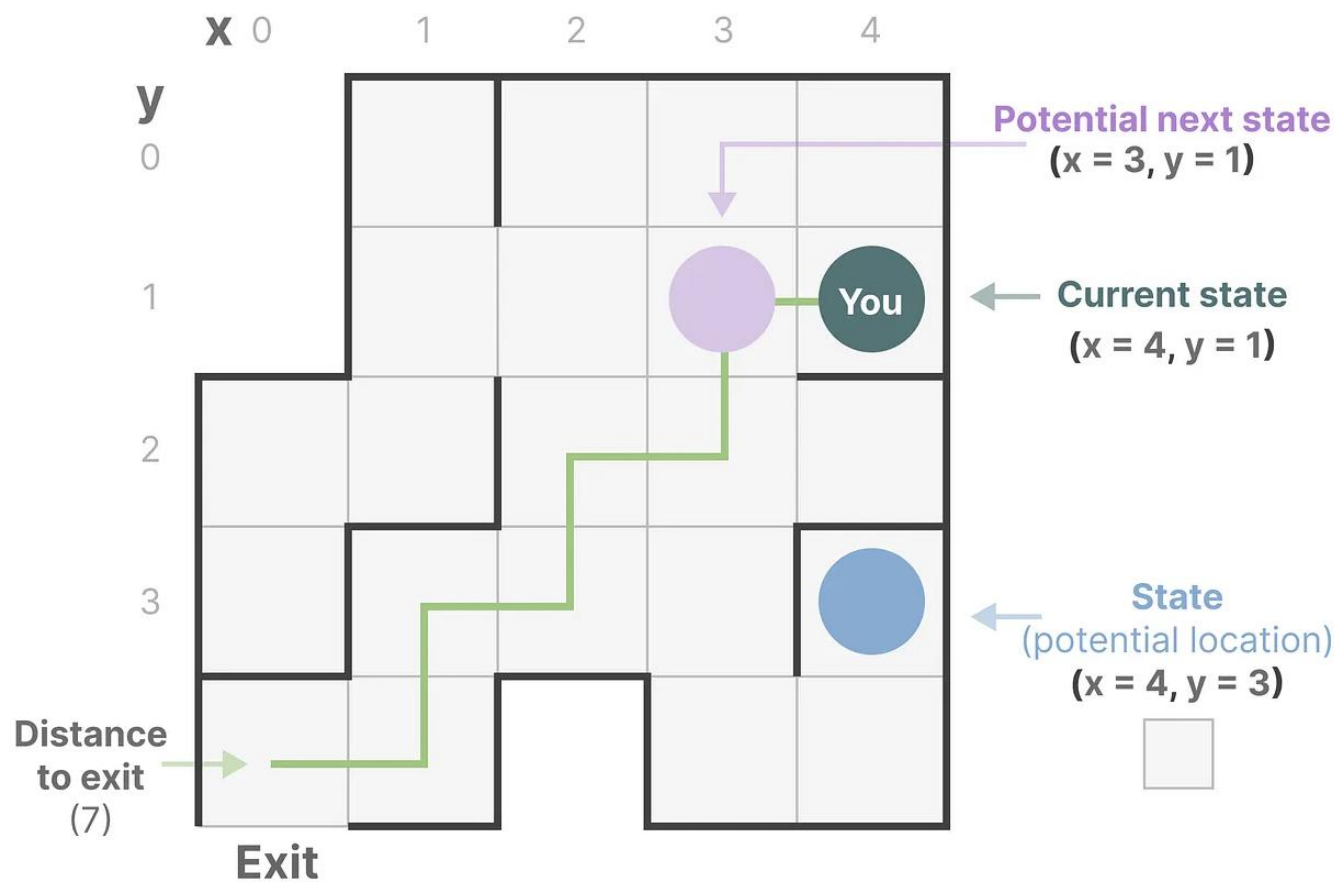
```
Mamba(  
  (embedding): Embedding(50280, 1024)  
  (layers): ModuleList(  
    (0-47): 48 x ResidualBlock(  
      (mixer): MambaBlock(  
        (in_proj): Linear(in_features=1024, out_features=4096, bias=False)  
        (conv1d): Conv1d(2048, 2048, kernel_size=(4,), stride=(1,), padding=(3,), groups=2048)  
        (x_proj): Linear(in_features=2048, out_features=96, bias=False)  
        (dt_proj): Linear(in_features=64, out_features=2048, bias=True)  
        (out_proj): Linear(in_features=2048, out_features=1024, bias=False)  
      )  
      (norm): RMSNorm()  
    )  
  )  
  (norm_f): RMSNorm()  
  (lm_head): Linear(in_features=1024, out_features=50280, bias=False)  
)
```

Mamba架构的方法思路：

- 选择性状态空间：Mamba 引入选择性状态空间模型，更高效、更有效地捕捉长序列中的相关信息。
- 线性时间复杂性：Mamba的运行时间与序列长度成线性关系。这一特性使其特别适用于超长序列的任务，而传统的模型在这方面会很吃力。

Transformer每次生成新标记时，模型都需要重新计算整个序列的注意力。这意味着即使已经生成了一部分文本，模型也需要考虑到整个序列的上下文，重新评估每个标记之间的关系。这种计算开销随着序列长度的增加而加剧，影响生成效率。





状态空间是一种数学工具，用来描述一个系统的所有可能状态。

在一个问题中，状态空间包含了系统所能达到的所有可能情形。

以迷宫为例，状态空间就像是迷宫中所有位置的地图，每个位置代表着一个状态。

状态空间表示则是对这张地图的简化，它不仅告诉你当前所处的位置，还会指示你能去的下一个位置以及如何从一个状态转移到另一个状态（例如，向右或向左）。

通过这种方式，状态空间模型能够有效地表达复杂系统的动态过程和决策路径。

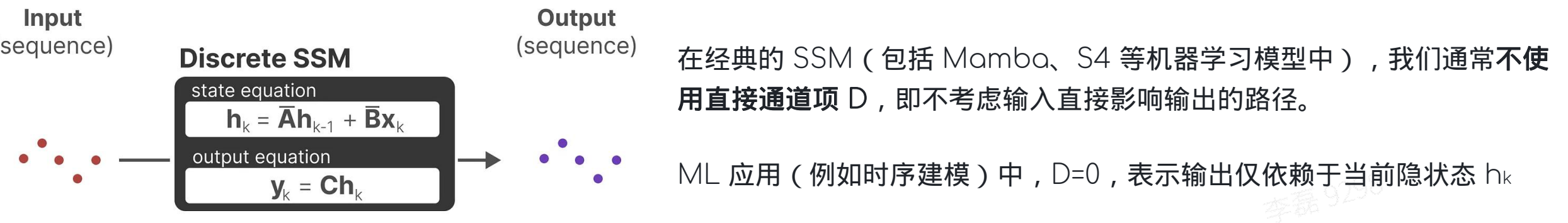
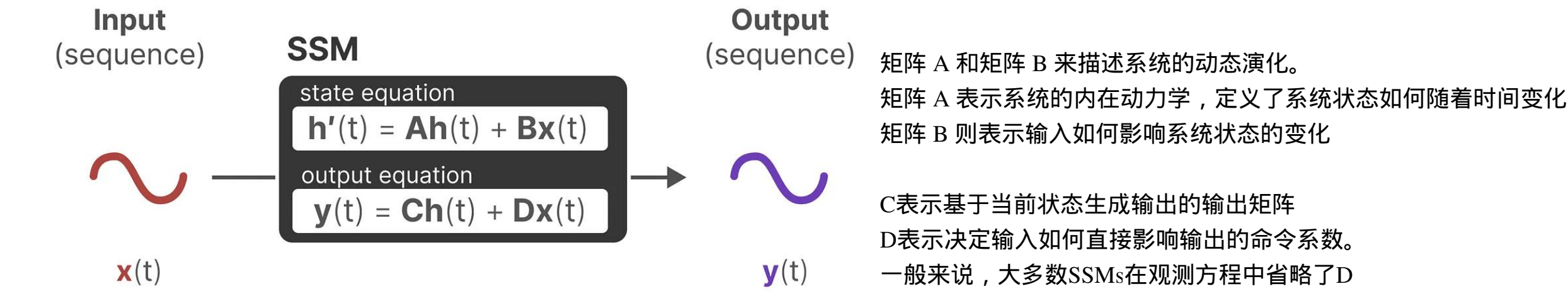
状态向量：包含所有必要信息的数学向量，用于全面描述系统的当前状态，并为后续的状态转移提供基础。

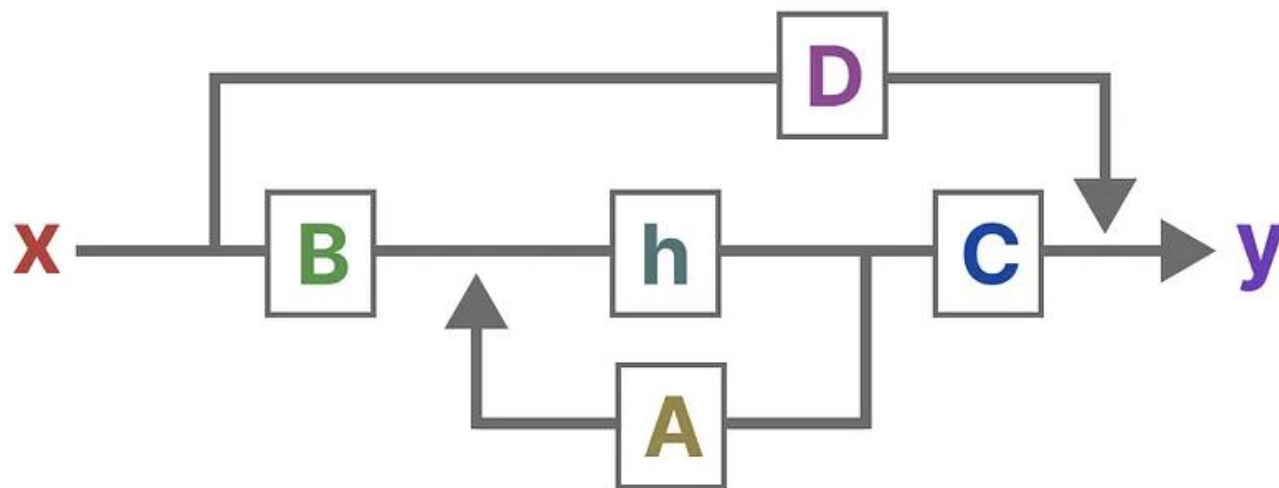
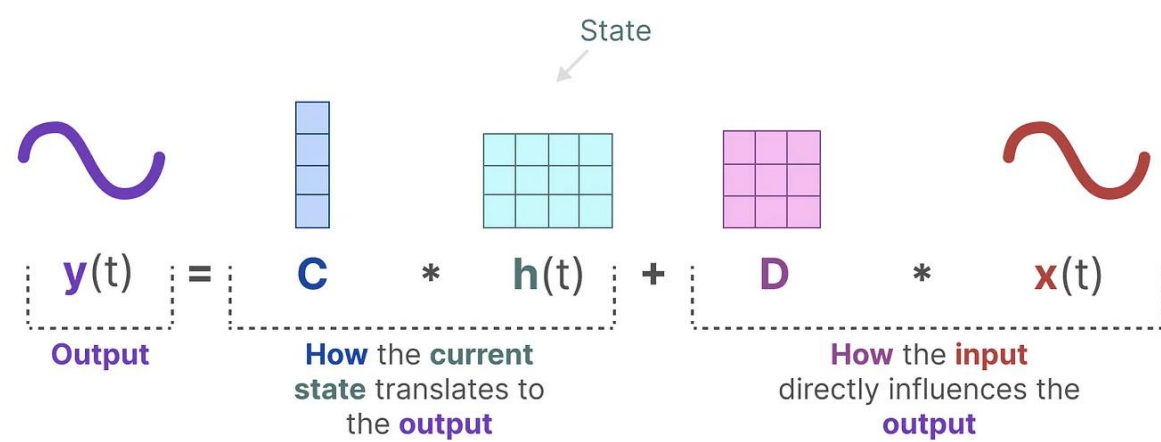
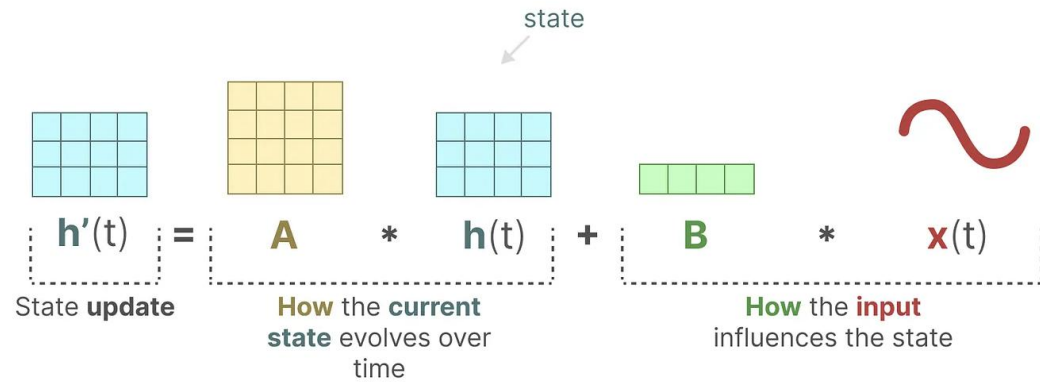
处理文本时，每个词语或句子的嵌入向量实际上就是该词或句子的“状态向量”。

在语言模型中，当前句子的嵌入向量也可以用来表示该句子的“状态”。

神经网络中，系统的“状态”通常指的是其隐藏状态。在大型语言模型中，隐藏状态是生成下一个标记的关键，它携带了关于输入序列的上下文信息。每次生成新标记时，模型会根据当前的隐藏状态来决定最可能的下一个标记，确保生成的文本与上下文一致。

状态空间模型（SSMs）是一类通过描述状态的表示来做出预测的模型。现代的状态空间模型能够处理连续输入序列，并基于这些连续输入预测输出序列。





$$h_k = \bar{A}h_{k-1} + \bar{B}x_k,$$

$$y_k = Ch_k,$$

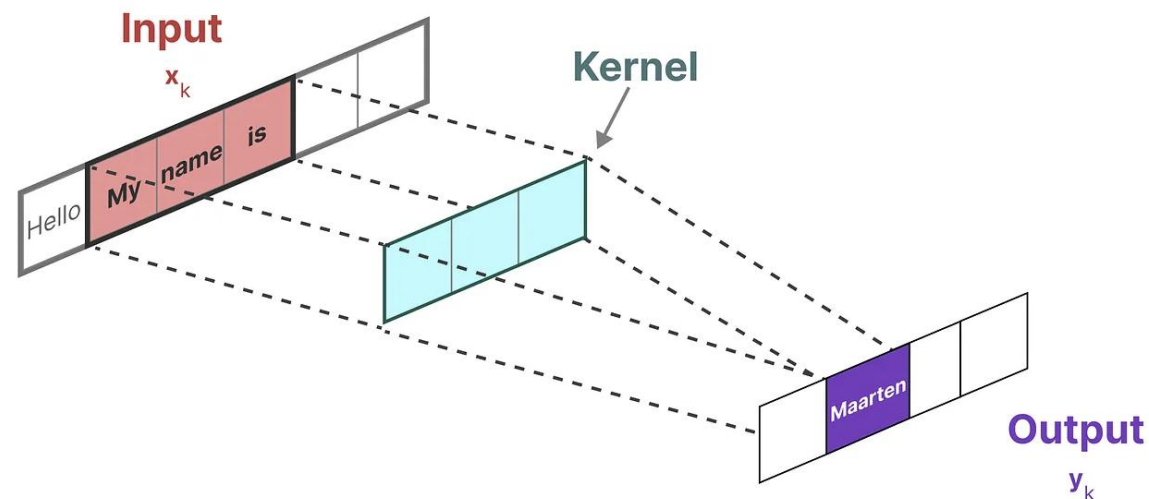
$$y_0 = C\bar{A}^0\bar{B}x_0,$$

$$y_1 = C\bar{A}^1\bar{B}x_0 + C\bar{A}^0\bar{B}x_1,$$

$$y_2 = C\bar{A}^2\bar{B}x_0 + C\bar{A}^1\bar{B}x_1 + C\bar{A}^0\bar{B}x_2,$$

.....

$$y_k = C\bar{A}^k\bar{B}x_0 + C\bar{A}^{k-1}\bar{B}x_1 + \dots + C\bar{A}^1\bar{B}x_{k-1} + C\bar{A}^0\bar{B}x_k.$$



Kernel



↓ Multiply



↓ Sum



$$y_0 = C\bar{B}x_0$$

Input

(x_k)

Output

(y_k)

Kernel

$\overline{C\overline{A}^2\overline{B}}$ $\overline{C\overline{A}\overline{B}}$ $\overline{C\overline{B}}$

↓ ↓ ↓ Multiply

Input

(x_k)



Output

(y_k)



$y_1 = \overline{C\overline{A}\overline{B}}x_0 + \overline{C\overline{B}}x_1$

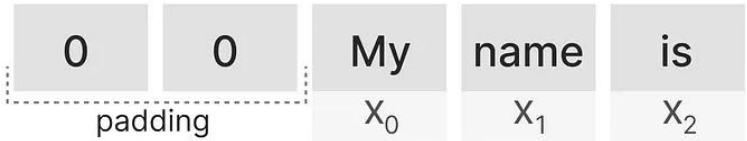
Kernel

$\overline{C\overline{A}^2\overline{B}}$ $\overline{C\overline{A}\overline{B}}$ $\overline{C\overline{B}}$

↓ ↓ ↓ Multiply

Input

(x_k)

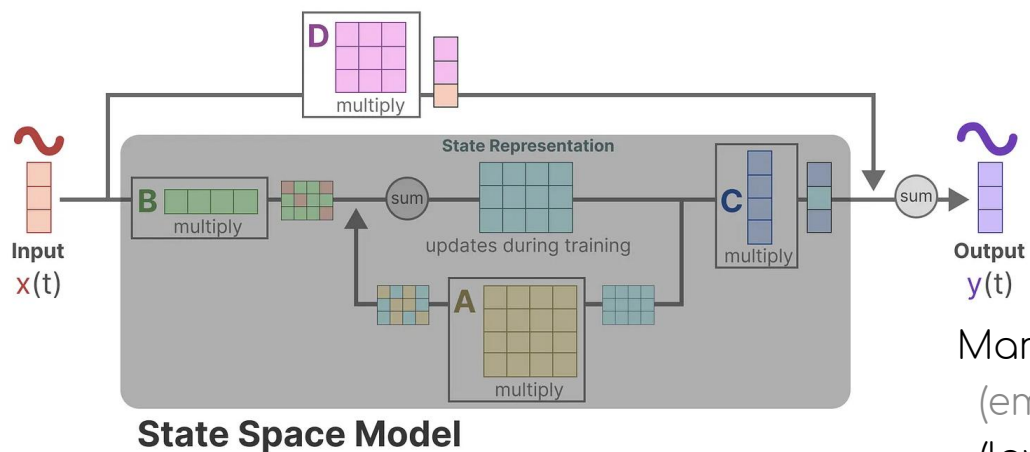


Output

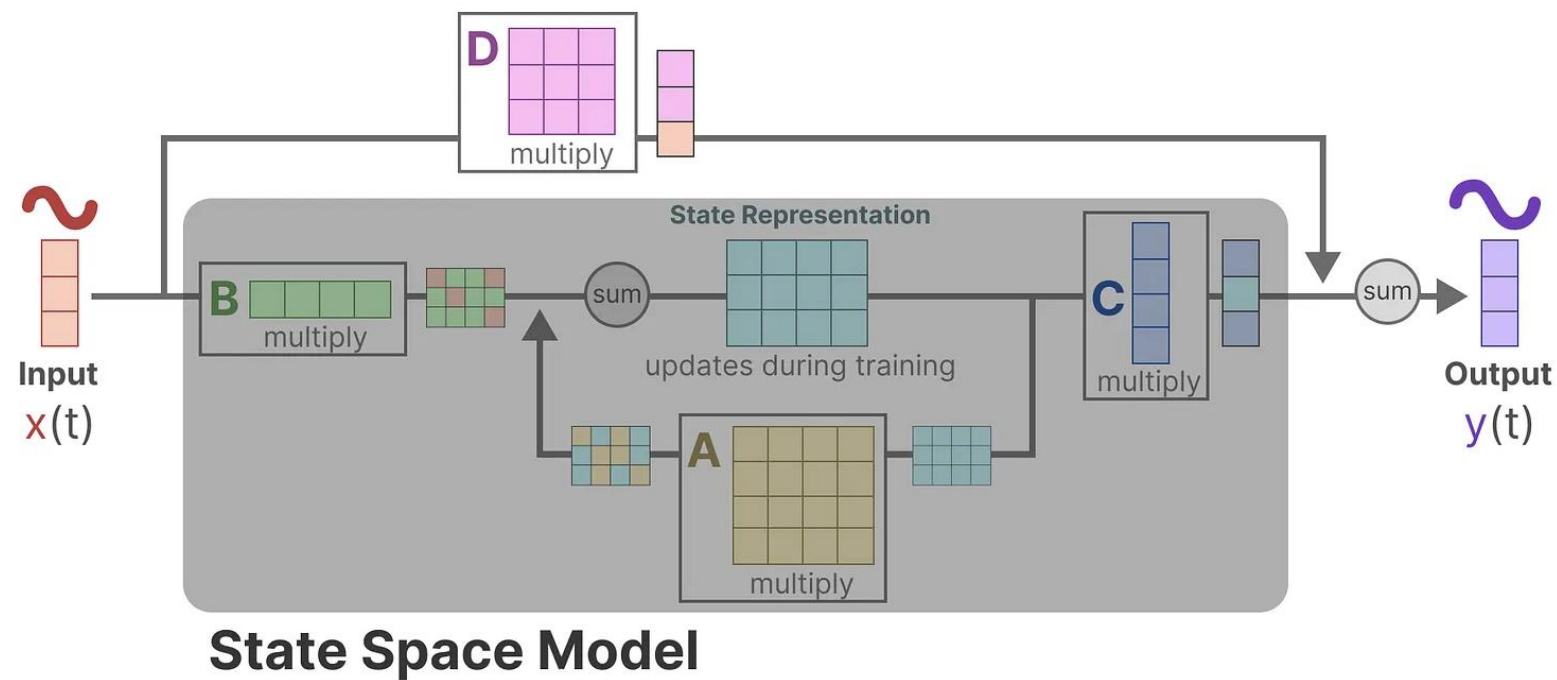
(y_k)



$y_2 = \overline{C\overline{A}^2\overline{B}}x_0 + \overline{C\overline{A}\overline{B}}x_1 + \overline{C\overline{B}}x_2$

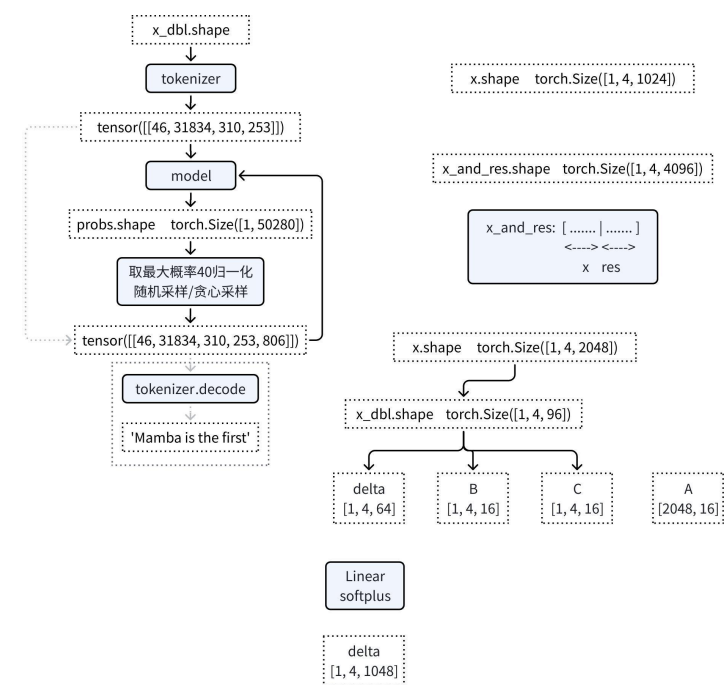


```
Mamba(
  (embedding): Embedding(50280, 1024)
  (layers): ModuleList(
    (0-47): 48 x ResidualBlock(
      (mixer): MambaBlock(
        (in_proj): Linear(in_features=1024, out_features=4096, bias=False)
        (conv1d): Conv1d(2048, 2048, kernel_size=(4,), stride=(1), padding=(3,), groups=20)
        (x_proj): Linear(in_features=2048, out_features=96, bias=False)
        (dt_proj): Linear(in_features=64, out_features=2048, bias=True)
        (out_proj): Linear(in_features=2048, out_features=1024, bias=False)
      )
      (norm): RMSNorm()
    )
  )
  (norm_f): RMSNorm()
  (lm_head): Linear(in_features=1024, out_features=50280, bias=False)
)
```



```
output = self.mixer(self.norm(x)) + x
```

```
self.mixer = MambaBlock(args)
```



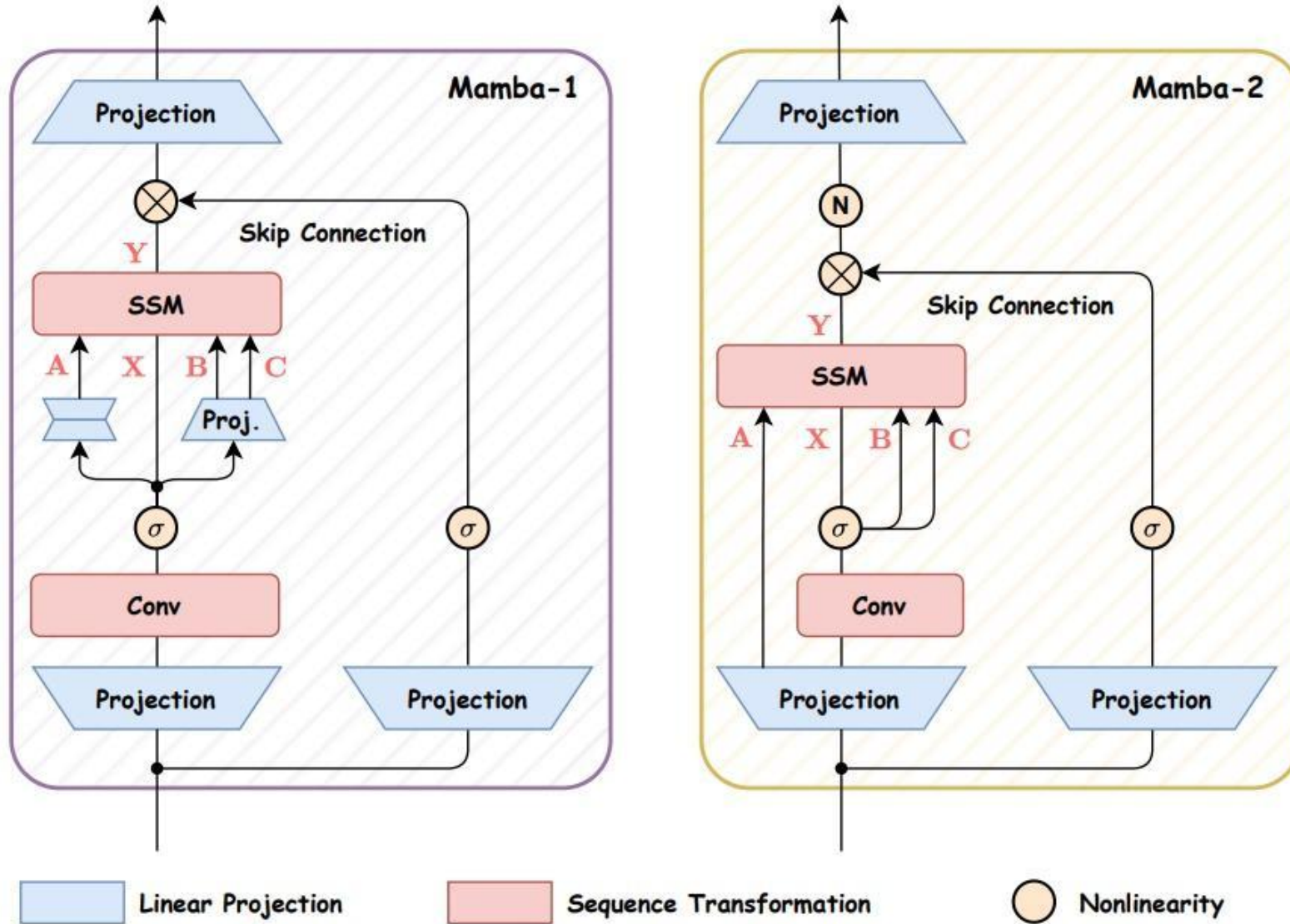
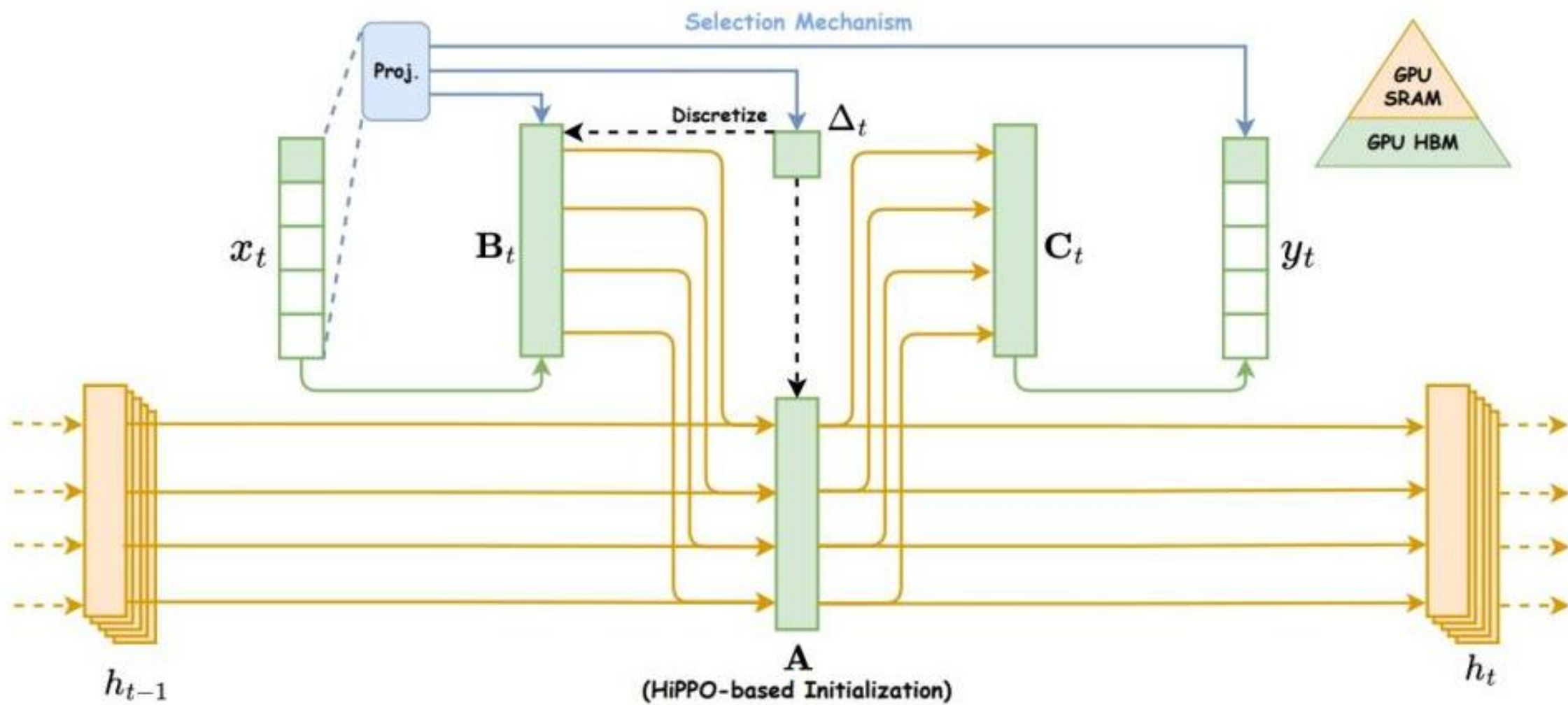
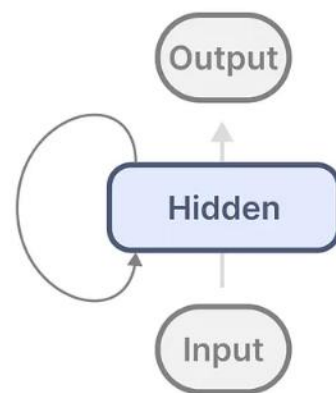


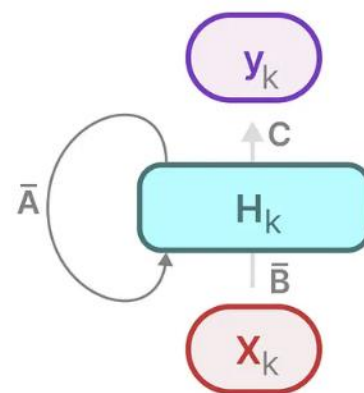
Fig. 4. The block architectures of Mamba-1 and Mamba-2.



这种表示可能看起来有点熟悉！我们可以用之前对 RNN 进行处理的方式来处理它。

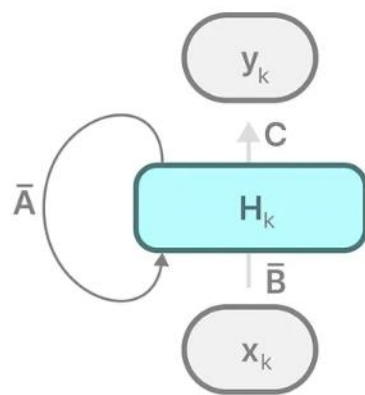


RNN

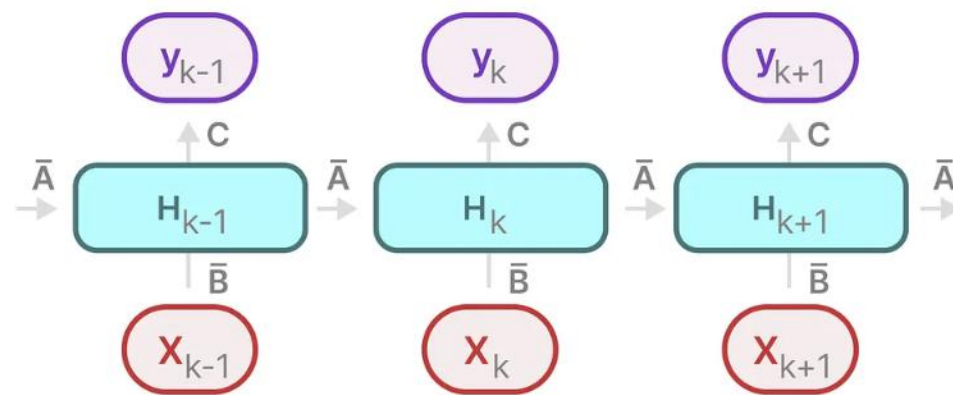


SSM
(Recurrent)

我们可以将其展开（或展开）如下：



SSM
(Recurrent)



SSM
(Recurrent + Unfolded)

